DevOpsCon

3 – 6 December 2018
Munich

DOCKER & KUBERNETES WHITEPAPER 2018

www.devopsconference.de

Services and Stacks in the Cluster

Continuous Deployment with Docker Swarm

In the DevOps environment, Docker can no longer be reduced to only a container runtime. An application that is divided into several microservices has greater orchestration requirements instead of simple scripts. For this, Docker has introduced the service abstraction Docker Swarm to help orchestrate containers across multiple hosts.

By Tobias Gesellchen

Docker Swarm: The way to continuous deployment

Docker Swarm is available in two editions. As a standalone solution, the older variant requires a slightly more complex set-up with its own key-value store. The newer variant, also called "swarm mode", has been part of the Docker Engine since Docker 1.12 and no longer needs a special set-up. This article only deals with swarm mode as it is recommended by the official authorities and has been developed more intensively. Before we delve deeper into the Swarm, let's first look at what Docker Services are and how they relate to the well-known Docker Images and containers.

Docker Swarm: From containers to tasks

Traditionally, developers use Docker Images as a means of wrapping and sharing artifacts or applications. The method of using complete Ubuntu images as Docker Images (which was initially common) has already been overtaken by minimal binaries in customized operating systems like Alpine Linux. The interpretation of a container has changed from virtual machine replacement to process capsule. The trend towards minimal Docker Images enables greater flexibility and better resource conservation. This way, both storage and network are less stressed, and additionally provide smaller images with fewer features, which leads to a smaller attack surface. Therefore, starting up containers is faster, and you have better dynamics. With this dynamic, a microservice stack is really fun to use and even paves the way for projects like Functions as a Service.

However, Docker Services don't obsolete containers, but complement configuration options, such as the desired number of replicas, deployment constraints (e. g., do not set up proxy on the database node) or update policies. Containers with their service-specific properties are called "tasks" in the context of services. Tasks are therefore the smallest unit that runs within a service. Since containers are not aware of the Docker Swarm and its service abstraction, the task acts as a link between swarm and container.

You can set up a service, for example based on the image nginx:alpine, with three replicas so that you receive a fail-safe set-up. The desired three replicas express themselves as three tasks and thus as containers, which are distributed for you by Docker Swarm on the available set of Swarm Nodes. Of course, you can't achieve fail-safe performance just by tripling the containers. Rather, Docker Swarm now knows your desired target configuration and intervenes accordingly if a task or node should fail.

Dive right in

In order to make the theory more tangible, we go through the individual steps of a service deployment. The one prerequisite is a current Docker release; I am using the current version 17.07 on Docker for Mac. Incidentally, all of the examples can be followed on a single computer, but in a productive environment, they are only useful across different nodes. All aspects of a production environment can be found in the official documentation. This article will only be able to provide selected hints.

The Docker Engine starts by default with disabled Swarm Mode. To enable it, enter on the console: *docker swarm init*.

Docker acknowledges this command by confirming that the current node has been configured as a manager. If you have already switched the Docker Engine to Swarm Mode before, an appropriate message will be displayed.

Docker Swarm differentiates between managers and workers. Workers are available purely for deploying tasks, while managers also maintain the Swarm. This includes continuously monitoring the services, comparing them with the desired target state and possibly reacting to deviations. Three or even five nodes are set up as managers in a production environment to ensure that the Swarm manager retains its ability to make decisions in the event of a manager's failure. These maintain the global cluster state via raft log, so that if the leader manager fails, one of the other managers assumes the role of a leader. If more than half of the managers fail, an incorrect cluster state can no longer be corrected. However, tasks that are already running on intact nodes remain in place.

In addition to the success message, the command entered above also displays a template for adding worker nodes. Workers need to reach the manager at the IP address at the very end of the command. This can be difficult for external workers under Docker for Mac or Docker for Windows because on these systems, the engine runs in a virtual machine that uses internal IP addresses.

The examples become a bit more realistic if we start more worker nodes locally next to the manager. This can be done very easily with Docker by starting one container per worker in which a Docker Engine is running. This method even allows you to try different versions of the Docker Engine without having to set up a virtual machine or a dedicated server.

In our context, when services are started on individual workers, it is also relevant that each worker must pull the required images from the Docker Hub or another registry. With the help of a local registry mirror, these downloads can be slightly optimized. That's not everything: we set up a local registry for locally-built images, so that we aren't forced to push these private images to an external registry such as the Docker Hub for deployment. How to set up the complete setup using scripts has already been described.

To simplify the set-up even further, Docker Compose is available. You can find a suitable docker-compose.yml, on GitHub, which starts three workers, a registry and a registry mirror. The following commands set up the necessary environment to help you understand the examples described in the article.

```
git clone https://github.com/gesellix/swarm-examples.git
cd swarm-examples
swarm/01-init-swarm.sh
swarm/02-init-worker.sh
```

All other examples can also be found in the named repository. Unless described otherwise, the commands are executed in its root directory.

The first service

After the local environment is prepared, you can deploy a service. The nginx as a triple replica can be set up as follows:

```
docker service create \
  --detach=false \
  --name proxy \
  --constraint node.role==worker \
  --replicas 3 \
  --publish 8080:80 \
  nginx:alpine
```

Most options such as -name or -publish should not be a surprise; they only define an individual name and configure port mapping. In contrast to the usual docker run, -replicas 3 directly defines how many instances of the nginx are to be started, and -constraint=… requires that service tasks may only be started on worker nodes and not on managers. Additionally, -detach=false allows you to monitor the service deployment. Without this parameter, or -detach=true, you can continue working directly on the console and the service is deployed in the background.

The command instructs the Docker Engine to download the desired image on the individual workers, create tasks with the individual configuration, and start the containers. Depending on the network bandwidth, the initial download of the images takes the longest. The start time of the containers depends on the concrete images or the process running in the container.

If you want to run a service on each active node instead of a specific number of replicas, the service can be started with –mode global. If you subsequently add new node workers to the Swarm, Docker will automatically extend the global-Service to the new nodes. Thanks to this kind of configuration, you no longer have to manually increase the number of replicas by the number of new nodes. Commands such as docker service ls and docker service ps proxy show you the current status of the service or its tasks after deployment. But even with conventional commands like docker exec swarm_worker2_1 docker ps, you will find the instances of nginx as normal containers. You can download the standard page of nginx via browser or curl at http://localhost:8080.

Before we look at the question of how three containers can be reached under the same port, let's look at how Docker Swarm restores a failed task. A simple docker kill swarm_worker2_1, which removes one of the three containers, is all that is needed for the Swarm to create a new task. In fact, this happens so fast that you should already see the new container in the next docker service ps proxy. The command shows you the task history, i. e. also the failed task. Such automatic self-healing of failed tasks can probably be regarded as one of the core features of container managers. With swarm/02-init-worker. sh you can restart the just stopped worker.

Docker Swarm allows you to configure how to react to failed tasks. For example, as part of a service update, the operation may be stopped, or you may want to roll back to the previous version. Depending on the context, it makes sense to ignore sporadic problems so that the service update is attempted with the remaining replicas.

Load Balancing via Ingress Network

Now, we return to the question of how the same port is bundled on three different containers in one service. In fact, the service port is not tied to the physical network interface with conventional means per container, but the Docker Engine sets up several indirections that route incoming traffic over virtual networks or bridges. Specifically, the Ingress Network was used for the request at http://localhost:8080, which can route packages to any service IP as a cross-node overlay network. You can also view this network with docker network ls and examine it in detail with docker network inspect ingress.

Load Balancing is implemented at a level that also enables the uninterrupted operation of frontend proxies. Typically, web applications are hidden behind such proxies in order to avoid exposing the services directly to the Internet. In addition to a greater hurdle for potential attackers, this also offers other advantages, such as the ability to implement uninterrupted continuous deployment. Proxies form the necessary intermediate layer to provide the desired and available version of your application.

The proxy should always be provided with security corrections and bugfixes. There are various mechanisms to ensure that interruptions at this level are kept to a minimum. When using Docker Services, however, you no longer need special devices. If you shoot down one instance of the three nginx tasks as shown above, the other two will still be accessible. This happens not only locally, but also in a multi-node Swarm. The only requirement is a corresponding swarm of docker engines and an intact ingress network.

Deployment via serviceupdate

Similar to the random or manual termination of a task, you can also imagine a service update. As part of the service update, you can customize various properties of the service. These include the image or its tag, you can change the container environment, or you can customize the externally accessible ports. In addition, secrets or configs available in the Swarm can be made available to a service or withdrawn again. Describing all the options here would go beyond the scope of the article, the official documentation will help you in detail. The following example shows you how to add an environment variable FOO and how to influence the process flow of a concrete deployment:

```
docker service update \
  --detach=false \
  --env-add FOO=bar \
  --update-parallelism=1 \
  --update-order=start-first \
  --update-delay=10s \
  --update-failure-action=rollback \
  proxy
```

At first glance, the command looks very complex. Ultimately, however, it only serves as an example of some options that you can tailor to your needs with regard to updating. In this example, the variable in the containers is supplemented by -env-add. This is done step-by-step across the replicas (-update-parallelism=1), whereby a fourth instance is started temporarily before an old version is stopped (-update-order=start-first). Between each task update, there is a delay of ten seconds (-update-delay=10s) and in case of an error, the service is rolled back to the previous version (-update-failure-action=rollback).

In a cluster of swarm managers and workers, you should avoid running resource-hungry tasks on the manager nodes. You probably don't want to run the proxy on the same node as the database. To map such rules, Docker Swarm allows configuring Service constraints. The developer expresses these constraints using labels. Labels can be added to or removed from the docker service create and via docker service update. Labels on services and nodes can be changed without even interrupting the task. You have already seen an ex-

DevOpsCon

Also visit this Session:

Azure Container Registry – a Serverless Docker Registry-as-a-Service

Rainer Stropek (software architects/www.IT-Visions.de)

If you want to privately deliver your Docker images to your data centers or customers world-wide, you will need to run your own registry. Running it yourself or using IaaS in the cloud for that means investing a lot of effort. Ready-made registries in the cloud are an alternative. Long-time Azure MVP and Microsoft Regional Director Rainer Stropek spends this session showing you how to setup, configure and use the serverless Container Registry in Microsoft's Azure cloud.

ample above as node. role==worker, for more examples see the official documentation.

Imagine that you not only have to maintain one or two services, but maybe ten or twenty different microservices. Each of these services would now have to be deployed using the above commands. Service abstraction takes care of distributing the concrete replicas to different nodes.

Individual outages are corrected automatically, and you can still get an overview of the health of your containers with the usual commands. As you can see, the command lines still take an unpleasant length. We have not yet discussed how different services can communicate with each other at runtime and how you can keep track of all your services.

Inter-service-communication

There are different ways to link services. We have already mentioned Docker's so-called overlay networks, which allow node-spanning (or node-ignoring) access to services instead of concrete containers or tasks. If you want the proxy configured above to work as a reverse proxy for another service, you can achieve this with the commands from Listing 1.

After the creation of an overlay network app, a new Service whoami is created in this network. Then the proxy from the example above is also added to the network. The two services can now reach each other using the service name. Ports do not have to be published explicitly for whoami, but docker makes the ports declared in the image via EXPOSE accessible within the network. In this case, the whoami-Service listens within the shared network on port 80.

All that is missing now is to configure the proxy to forward incoming requests to the whoami-Service. The nginx can be configured like in Listing 2 as a reverse proxy for the whoami-Service.

Listing 1

```
docker network create \
  --driver overlay \
  app

docker service create \
  --detach=false \
  --name whoami \
  --constraint node.role==worker \
  --replicas 3 \
  --network app \
  emilevauge/whoami

docker service update \
  --detach=false \
  --network-add app \
  proxy
```

The matching Dockerfile is kept very simple, because it only has to add the individual configuration to the standard image:

```
FROM nginx:alpine
RUN rm /etc/nginx/conf.d/*
COPY backend.conf /etc/nginx/conf.d/
```

The code can be found in the GitHub repository mentioned above. The following commands build the individual nginx image and load it into the local registry. Afterwards, the already running nginx is provided with the newly created image via service update:

```
docker build -t 127.0.0.1:5000/nginx -f nginx-basic/Dockerfile nginx-basic
docker push 127.0.0.1:5000/nginx

docker service update \
  --detach=false \
  --image registry:5000/nginx \
  proxy
```

The service update shows that the image name instead of 127.0.0.1 is now registry as the repository host. This is necessary because the image should be loaded from the worker's point of view and they only know the local registry under the name registry. However, the manager cannot resolve the registry hostname, thereby not verifying the image and therefore warns against potentially differing images between the workers during the service update.

After a successful update you can check via curl http://localhost:8080 if the proxy is reachable. Instead of the nginx default page, the response from the whoami-Service should now appear. This response always looks a bit different for successive requests, because the round-robin loadbalancing mode in Docker always redirects you to the next task. The best way to recognize this is the changed hostname or IP. With docker service update -replicas 1 whoami or docker service update -replicas 5 whoami you can easily scale up or down the service, while the proxy will always use one of the available instances.

Listing 2

```
upstream backend {
  server whoami;
}

server {
  listen 80;

  location / {
    proxy_pass http://backend;
    proxy_connect_timeout 5s;
    proxy_read_timeout 5s;
  }
}
```

Figure 1 shows an overview of the current Swarm with three worker nodes and a manager. The dashed arrows follow the request on http://localhost:8080 through the two overlay networks ingress and app. The request first lands on the nginx task proxy. 2, which then acts as reverse proxy and passes the request to its upstream backend. Like the proxy, the backend is available in several replicas, so that the task whoami. 3 is accessed at worker 3 for the specific request.

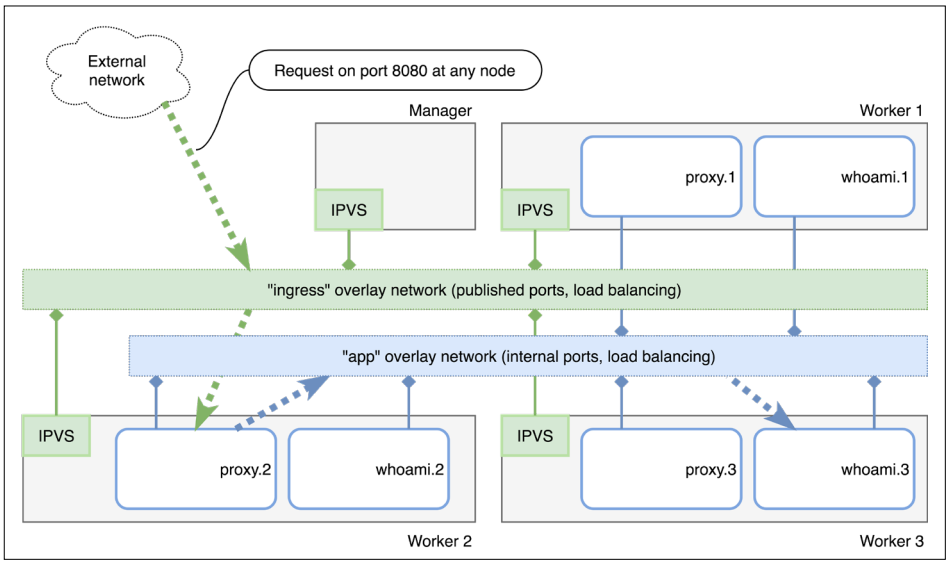

Fig. 1: A request on its way through overlay networks

You have now learned how existing services can be upgraded without interruption, how to react to changing load using a one-liner, and how overlay networks can eliminate the need to publish internal ports on an external interface. Other operational details are just as easy to handle, e.g. when the Docker Engines, worker or managers need to be updated or individual nodes need to be replaced. For these use cases, see the relevant notes in the documentation.

For example, a node can be instructed to remove all tasks via docker node update -availability=drain. Docker will then take care of clearing the node virtually empty, so that you can carry out maintenance work undisturbed and without risk. With docker swarm leave and docker swarm join you can always remove or add workers and managers. You can obtain the necessary join tokens from one of the managers by calling docker swarm join-token worker or docker swarm join-token manager.

Docker Stack

As already mentioned, it is difficult to keep track of a growing service landscape. In general, Consul or similar tools are suitable for maintaining a kind of registry that provides you with more than just an overview. Tools such as Portainer come with support for Docker Swarm and dashboards that give you a graphical overview of your nodes and services.

Docker offers you a slim alternative in the form of Docker Stack. As the name suggests, this abstraction goes beyond the individual services and deals with the entirety of your services, which are closely interlinked or interdependent. The technological basis is nothing new, because it reuses many elements of Docker Compose. Generally speaking: Docker Stack uses Compose's YAML format and complements the Swarm-specific properties for service deployments. As an example, you can find the stack for the manually created services under nginx-basic/docker-stack.yml. If you want to try it instead of manually setting up services, you must first stop the proxy to release port 8080. The following commands ensure a clean state and start the complete stack:

```
docker service rm proxy whoami
docker network rm app

docker stack deploy --compose-file nginx-basic/docker-stack.yml example
```

The docker stack deploy command receives the desired stack description via -compose-file. The name example serves on the one hand as an easily recognizable reference to the stack and internally as a means of namespacing the various services. Docker now uses the information in the docker-stack.yml to generate virtually the equivalent of the docker service create … commands internally and sends them to the Docker Engine.

Compared to Compose, there are only some new blocks in the configuration file – the ones under deploy, which, as already mentioned, define the Swarm-specific properties. Constraints, replicas and update behavior are configured appropriately to the command line parameters. The documentation contains details and other options that may be relevant to your application.

The practical benefit of the stacks is that you can now check in the configuration to your VCS and therefore have complete and up-to-date documentation on the setup of all connected services. Changes are then reduced to editing this file and the repeated docker stack deploy -compose-file nginx-basic/docker-stack.yml example. Docker checks on every execution of the command if there are any discrepancies between the YAML content and the services actually deployed and corrects them accordingly via internal docker service update. This gives you a good overview of your stack. It is versioned right along the source code of your services and you need to maintain far less error-prone scripts. Since the stack abstraction is a purely client-

side implementation, you still have full freedom to perform your own actions via manual or scripted docker servicecommands.

If the constant editing of docker-stack.yml seems excessive in the context of frequent service updates, consider variable resolution per environment. The placeholder NGINX_IMAGE is already provided in the example stack. Here is the relevant excerpt:

```
...
services:
  proxy:
    image: „${NGINX_IMAGE:-registry:5000/nginx:latest}“
...
```

With an appropriately prepared environment, you can deploy another nginx image without first editing the YAML file. The following example changes the image for the proxy back to the default image and updates the stack:

```
export NGINX_IMAGE=nginx:alpine
docker stack deploy --compose-file nginx-basic/docker-stack.yml example
```

The deployment now runs until the individual instances are updated. Afterwards, a curl http://localhost:8080 should return to the nginx default page. The YAML configuration of the stack thus remains stable and is adapted only by means of environment variables.

The resolution of the placeholders can be done at any position. In practice, it would therefore be better to keep only the image tag variable instead of the complete image.

```
...
services:
  proxy:
    image: „nginx:${NGINX_VERSION:-alpine}“
...
```

Removing a complete stack is very easy with docker stack rm example.

Please note: all services will be removed without further enquiry. On a production system, the command can likely be considered dangerous, but it makes handling services for local set-ups and on test stages very convenient.

As mentioned above, the stack uses namespacing based on labels to keep different services together, but it works with the same mechanisms as docker service… commands. Therefore, it is up to you to supplement a stack initially deployed via docker stack deploy with docker service update during operation.

Secrets and service-configs

Docker services and stack offer you more than only the management of tasks across different nodes. Secrets and configs can also be distributed more easily using Docker Swarm and are more securely stored in only those container file systems that you have authorized, compared to the environment variables recommended at https://12factor.net/.

Basically, Docker Secrets and Configs share the same concept. You first create objects or files centrally in Swarm via docker secret create… or docker config create…, which are stored internally by Docker – Secrets are encrypted beforehand. You give these objects a name, which you then use when you link them to services.

Based on the previous example with nginx and extracts from the official docker documentation, we can add HTTPS support. Docker Swarm mounts the necessary SSL certificates and keys as files in the containers. Secrets only end up in a RAM disk for security reasons. First, you need suitable certificates that are prepared in the repository under nginx-secrets/cert. If you want to update the certificates, a suitable script nginx-secrets/gen-certs.sh is available.

Docker Swarm allows up to 500 KB of content per secret, which is then stored as a file in /run/secrets/. Secrets are created as follows:

```
docker secret create site.key nginx-secrets/cert/site.key
docker secret create site.crt nginx-secrets/cert/site.crt
```

Configs can also be maintained similarly to secrets. By looking at the example of the individual nginx configuration from the beginning of the article, you will soon see that the specially built image will no longer be necessary. To configure the nginx, we use the configuration under nginx-secrets/https-only.conf and create it using Docker Config:

```
docker config create https.conf nginx-secrets/https-only.conf
```

First, you define the desired name of the config. Then you enter the path or file name, for the contents you want Docker to store in the Swarm. With docker secret ls and docker config ls you can find the newly created objects. Now all that's missing is the link between the service, and the Swarm Secrets and Config. For example, you can start a new service as follows. Note that the official nginx image is sufficient:

```
docker service create \
  --detach=false \
  --name nginx \
  --secret site.key \
  --secret site.crt \
  --config source=https.conf,target=/etc/nginx/conf.d/https.conf \
  --publish 8443:443 \
  nginx:alpine
```

In the browser you can see the result at https://localhost:8443, but you have to skip some warnings because of the self-issued Certification Authority of the server certificate. In this case the check is easier via command line:

```
curl --cacert nginx-secrets/cert/root-ca.crt https://localhost:8443
```

Secrets and configs are also supported in Docker Stack. To match the manual commands, the Secret or Config

Listing 3

```
version: „3.4“

services:
  proxy:
    image: „${NGINX_IMAGE:-nginx:alpine}“
    networks:
      - app
    ports:
      - „8080:80“
      - „8443:443“
    deploy:
      placement:
        constraints:
          - node.role==worker
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
      restart_policy:
        condition: any
    configs:
      - source: https.conf
        target: /etc/nginx/conf.d/https.conf
    secrets:
      - site.key
      - site.crt
  whoami:
    image: emilevauge/whoami:latest
    networks:
      - app
    deploy:
      placement:
        constraints:
          - node.role==worker
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
      restart_policy:
        condition: on-failure

networks:
  app:
    driver: overlay

configs:
  https.conf:
    file: ./https-backend.conf

secrets:
  site.key:
    file: ./cert/site.key
  site.crt:
    file: ./cert/site.crt
```

is also declared and, if necessary, created within the YAML file at the top level, while the link to the desired services is then defined for each service. Our complete example looks like shown in Listing 3 and can be deployed as follows:

```
cd nginx-secrets
docker stack deploy --compose-file docker-stack.yml https-example
```

Updating secrets or configs is a bit tricky. Docker cannot offer a generic solution for updating container file systems. Some processes expect a signal like SIGHUP when updating the configuration, others do not allow a reload, but have to be restarted. Docker therefore suggests to create new secrets or configs under a new name and replace them with the old versions by docker service update -config-rm -config-add…

Stateful services and volumes

If you want to set up databases via docker service, you will inevitably be asked how the data will survive a container restart. You are probably already familiar with volumes to address this challenge. Usually, volumes are connected very closely to a specific container, so that both are practically one unit. In a swarm with potentially moving containers, such a close binding can no longer be assumed – a container can always be started on another node where either the required volume is completely missing, is empty or even contains obsolete data. From data volumes in the order of several gigabytes upwards, it is no longer useful to copy or move volumes to other nodes. Of course, depending on the environment you have several possible solutions.

The basic idea is to select a suitable volume driver, which then distributes the data to different nodes or to a central location. Docker therefore allows you to select the desired driver and, if necessary, configure it when creating partitions. There are already a number of plug-ins that connect the Docker Engine to new Volume Drivers. The documentation shows an extensive selection of these plug-ins. You may find the specific NetApp or vSphere plug-ins in your environment appropriate. Alternatively, We recommend the REX-Ray plug-in for closer inspection, as it enjoys a good reputation in the community and it is quite platform-neutral.

Since the configuration and use of the different volume plug-ins and drivers is too specific for your specific environment, I will not include a detailed description here. Please note that you must use at least Docker 1.13 or in some cases even version 17.03. The necessary docker-specific commands can usually be reduced to two lines, which are listed as examples for vSphere in Listing 4.

In addition to installing the plug-in under an alias vsphere, the second step is to create the desired MyVolume volume. Part of the configuration is stored in the file system, while you can configure individual parameters by -o at the time of volume creation.

Proxies with true docker swarm integration

Using the example of nginx, it was very easy to define statically the known upstream services. Depending on the application and environment, you may need a more dynamic concept and want to change the combination of the services more often. In today's microservices environment, the convenient addition of new services marks a common practice. Unfortunately, the static configuration of a nginx or HAProxy will then feel a bit uncomfortable. But fortunately, there are already convenient alternatives, of which Træfik is probably the most outstanding. Plus, it comes with excellent docker integration!

Equivalent to the first stack with nginx, you will find the same stack with Træfik. Træfik needs access to a Swarm Manager's Docker Engine API to dynamically adapt its configuration to new or modified services. It is therefore placed on the manager nodes using deployment constraints. Since Træfik cannot guess certain service-specific settings, the relevant configuration is stored on the respective services through labels.

In our example, you can see how the network configuration (port and network) is defined, so the routing will still reach the service, even if it is in multiple networks. In addition, the traefik.frontend.rule defines for which incoming requests packages should be forwarded to the whoami service. Besides routing based on request headers, you can also use paths and other request elements as criteria. See the Træfik documentation for respective information.

Finally, there are more details on integration with Docker Swarm in the Swarm User Guide. The example stack is still missing the configuration for HTTPS support, but since Træfik comes with native integration for Let's Encrypt, We only have to refer to appropriate examples.

Listing 4

```
docker plugin install \
  --grant-all-permissions \
  --alias \
  vsphere vmware/docker-volume-vsphere:latest
docker volume create \
  --driver=vsphere \
  --name=MyVolume \
  -o size=10gb \
  -o vsan-policy-name=allflash
```

Conclusion

Docker Swarm offers even more facets than shown, which may become more or less relevant depending on the context. Functions such as scheduled tasks or pendants to cron jobs as services are often requested, but currently difficult to implement with built-in features. Nevertheless, compared to other container orchestrators, Docker Swarm is still neatly arranged and lean. There are only a few hurdles to overcome in order to quickly achieve useful results.

Docker Swarm takes care of many details as well as the configurable error handling, especially for Continuous Deployment. With Docker Swarm, you don't have to maintain your own deployment code and you even get some rudimentary load balancing for free. Several features such as autoscaling can be supplemented via Orbiter and adapted to your own needs. The risk of experimentation remains relatively low because Docker Swarm has little invasive effect on the existing infrastructure. In any case, it's fun to dive right in with Swarm – whether via command line, YAML-file or directly via Engine-API.

Tobias Gesellchen is developer at Europace AG and Docker expert, who likes to focus on DevOps cultural and engineering wise.

Top Docker Tips From 12 Docker Captains

Docker is great, but sometimes you need a few pointers. We asked 12 Docker captains their top hack for our favorite container platform. We got some helpful advice and specific instructions on how to avoid problems when using Docker. Read on to find out more!

DOCKER TIP #1

Ajeet Singh Raina is Senior Systems Development Engineer at DellEMC Bengaluru, Karnataka, India. @ajeetraina

How do you use Docker?

Ajeet Singh Raina: Inside DellEMC, I work as Sr. Systems Development Engineer and spend considerable amount of time playing around with datacenter solution. Hardly a day goes by without talking about Docker and its implementation. Be it a system management tool, test certification, validation effort or automation workflow, I work with my team to look at how Docker can simplify the solution and save enormous time of execution. Being part of Global Solution Engineering, one can find me busy talking about the possible proof of concepts around datacenter solution and finding the better way to improve our day to day job. Also, Wearing a Docker captain's hat, there is a sense of responsibility to help the community users, hence I spend most of time keeping close eyes on Slack community questions/discussions and contributing in terms of blog posts almost every week.

Raina's Docker Tip:

Generally, Docker service inspect outputs a huge JSON dump. It becomes quite easy to access individual properties using Docker Service Inspection Filtering & Template Engine. For example, if you want to list out the port which WordPress is using for specific service:

```
$docker service inspect -f '{{with index .Spec.EndpointSpec.Ports 0}}
{{.TargetPort}}{{end}}' wordpressapp
```

Output:

```
80
```

This will fetch just the port number out of huge JSON dump. Amazing, isn't it?

DOCKER TIP #2

Nick Janetakis is Docker Trainer and creator of www.diveintodocker.com. @nickjanetakis

How do you use Docker?

Nick Janetakis: I use Docker in development for all of my web applications which are mostly written in Ruby on Rails and Flask. I also use Docker in production for a number of projects. These are systems ranging from a single host deploy, to larger systems that are scaled and load balanced across multiple hosts.

Janetakis' Docker Tip:

Don't be afraid of using Docker. Using Docker doesn't mean you need you need to go all-in with every single high scalability buzz word you can think of. Docker isn't about deploying a multi-data center load balanced cluster of services with green / blue deploys that allow for zero down deploys with seamless continuous integration and delivery. Start small by using Docker in development, and try deploying it to a single server. There are massive advantages to using Docker at all levels of skill and scale.

DOCKER TIP #3

Gianluca Arbezzano is Page Reliability Engineer at InfluxData Italy. @gianarb

How do you use Docker?

Gianluca Arbezzano: I use Docker to ship applications and services like InfluxDB around big cloud services. The container allows me to ship the same application in a safe way. I use Docker a lot to create and manage environments. With Docker Compose I can start a fresh environment to run smoke tests or integration tests on a specific application in a very simple and easy way. I can put it in my pipeline and delivery process to enforce my release circle.

Arbezzano's Docker Tip:
Generally, Docker service inspect outputs a huge JSON dump. It becomes quite easy to access individual properties using Docker Service Inspection Filtering & Template Engine. For example, if you want to list out the port which WordPress is using for specific service:

```
docker run -it -p 8000:8000 gianarb/micro:1.2.0
```

DOCKER TIP #4

Adrian Mouat is Chief Scientist at Container Solutions. @adrianmouat

Docker & Kubernetes

Container technology is spreading like wildfire in the software world — possibly faster than any other technology before. But what are the key learnings so far? Have the initial assumptions about the way in which containers revolutionize both the development and deployment of software been verified or falsified? What are the challenges for using containers in production and where are we headed to? This track provides use cases and best practices for working with the likes of Docker, Kubernetes & Co.

How do you use Docker?

Adrian Mouat: My daily work is helping others with Docker and associated technologies, so it plays a big role. I also give a lot of presentations, often running the presentation software from within a container itself.

Mouat's Docker Tip:
I have a whole presentation of tips that I'll be presenting at DockerConEU! But if you just want one, it would be to set the `docker ps` output format. By default it prints out a really long line that looks messy unless your terminal takes up the whole screen. You can fix this by using the `–format` argument to pick what fields you're interested in:

```
docker ps –format \
"table {{.Names}}\\t{{.Image}}\\t{{.Status}}"
```

And you can make this the default by configuring it in your `.docker/config.json` file.

DOCKER TIP #5

Vincent De Smet works as DevOps Engineer at Honestbee, Singapore. @vincentdesmet

How do you use Docker?

Vincent De Smet: Docker adoption started out mainly in the CI/CD pipeline and from there on through staging environments to our production environments. At my current company, developer adoption (using containers to develop new features for existing web services) is still lacking as each developer has their own preferred way of working. Given containers are prevalent everywhere else and Docker tools for developers keep improving, it will only take time before developer will choose to adopt these into their daily workflow. I personally, as a DevOps engineer in charge of maintaining containerized production environments as well as improve developer workflows, troubleshoot most issues through Docker containers and use containers daily.

De Smet's Docker Tip:
Make sure to follow "Best practices for writing Dockerfiles" – these provide very good reasons on why you should do things a certain way and I see way too many existing Dockerfiles that do not follow these.

Anyone slightly more advanced with Docker will also gain a lot from mastering the Linux Alpine distribution and its package manager.

And if you're getting started, training.play-with-docker.com is an amazing resource

```
docker run -it -p 8000:8000 gianarb/micro:1.2.0
```

DOCKER TIP #6

Chanwit Kaewkasi is Docker Swarm Maintainer and has ported Swarm to Windows. @chanwit

How do you use Docker?

Chanwit Kaewkasi: I help companies in South-East Asia and Europe design and implement their application architectures using Docker, and deploy them on a Docker Swarm cluster.

Kaewkasi's Docker Tip:
`docker system prune -f` always make my day.

DOCKER TIP #7

Kendrick Coleman is Developer Advocate for {code} by Dell EMC. @kendrickcoleman

How do you use Docker?

Kendrick Coleman: Docker plays a role in my daily job. I am eager to learn the innards to find new corner cases. It makes me excited to know I can turn knobs to make applications work the way I want. There is a misconception that persistent applications can't or shouldn't run in containers. I'm proud that the team I work with builds tools to make running persistent application easy and seamless that can be integrated as a part of a tool chain.

DevOpsCon

Also visit this Session:

Shell Ninja: Mastering the Art of Shell Scripting

Roland Huß (Red Hat)

Unix shell scripts are our constant companions since the seventies, and although there have been many other contenders like Perl or Python, Shell scripts are still here, alive and kicking. With the rise of the container writing shell scripts becomes an essential skill again, as plain shell scripts are the least common denominator for every Linux container. Even we as developers in a DevOps world can not neglect shell scripting.In this hands-on session, we will see how we can polish our shell-fu. We will see how the best practices we all have learned and love when doing our daily coding can be transferred to the shell scripting. An opinionated approach to coding conventions will be demonstrated for writing idiomatic, modular and maintainable scripts. Integration tests for non-trivial Shell scripts are as essential as for our applications, and we will learn how to write them. These techniques and much more will be part of our ride through the world of Bash & Co. Come and enjoy some serious shell script coding, you won't regret it and will see that Shell coding can be fun, too.

Coleman's Docker Tip:
Start off easy. Always go for the low hanging fruit like a web server and make it work for you. Then take your single host and pick an orchestrator and use that to make your app resilient. After that, move to an application that uses persistent data. This allows you to progress and move all your applications off of VMs and into containers.

DOCKER TIP #8

John Zaccone works as Cloud Engineer and Developer Advocate at IBM. @JohnZaccone

How do you use Docker?

John Zaccone: Right now, I work at IBM as a developer advocate. I work with developers from other companies to help them improve their ability to push awesome business value to production. I focus on adopting DevOps automation, containers, and container orchestration as a big part of that process.

Zaccone's Docker Tip:
I organize a meetup where I interface with a lot of developers and operators who want to adopt Docker, but find that they either don't have the time or can't clearly define the business case for using Docker. My advice to companies (and this applies to all new technologies, not just Docker) is to allow developers some freedom to explore new solutions. Docker is a technology where the benefits are not 100% realized until you get your hands on it and understand exactly how it will benefit you in your use case.

DOCKER TIP #9

Nicolas De Loof is Docker enthusiast at CloudBees. @ndeloof

How do you use Docker?

Nicolas De Loof: For my personal use I rely on Docker for various tests, so I ensure I have a reproducible environment I can share with others, as well as prevent impacts on my workstation. My company also offers a Docker based elastic CI/CD solution "CloudBees Jenkins Enterprise" and as a Docker expert I try to make it adopt best Docker features.

De Loof's Docker Tip:
Considering immutable infrastructure, there's many middleware who use filesystem as a cache, and on might want to avoid making this persistent. So I like to constrain them running as a read-only container (docker run –read-only) to know exactly where they need to access filesystem, then to create a volume for the actual persistent data directory and a tmpfs for everything else, typically caches or log files.

DOCKER TIP #10

Lorenzo Fontana is DevOps expert at Kiratech. @fntlnz

How do you use Docker?

Lorenzo Fontana: My company is writing an open source for Docker and other containerization technologies, I'm also daily involved in Docker doing mainly reviews on issues and PRs. I do a lot of consultancy to help companies using Containers and Docker by reflection. I used Docker for a while to spawn GUI software on my computer, and then I switched to systemd-nspawn. In the future, I'll probably go to runc.

Fontana's Docker Tip:
Not many people already know about multi staged builds, another cool thing is the fact that now Docker handles configs and secrets. Also a lot happens in the implementation, just get one project under the Docker or the Moby organizations on GitHub, there are a lot of implemented things that can open your eyes on how things works.

DOCKER TIP #11

Brian Christner is Cloud Advocate and Cloud architect at Swisscom. @idomyowntricks

How do you use Docker?

Brian Christner: I personally use Docker for every new project I'm working on. My personal blog runs Docker and the monitoring projects I'm working on to creating applications for IoT on RasperryPi's. At work, Docker is being used across several teams. We use it to provision our Database as a Service offerings and for development purposes. It is very versatile and used across multiple verticals within our company. Here is one of our use cases that is on Docker's website – "Swisscom goes from 400 vms to 20 vms, maximizing infrastructure efficiency with Docker".

De Loof's Docker Tip:
I share all my favorite tips via my blog.

DOCKER TIP #12

Antonis Kalipetis is CTO at SourceLair, a Docker based Online-IDE. @akalipetis

How do you use Docker?

Antonis Kalipetis: I use Docker for all sorts of things, as a tool to create awesome developer tools at SourceLair, in my local development workflow and for deploying production systems for our customers.

Kalipetis' Docker Tip:
My tip would be to always use Docker Swarm, or another orchestrator, for deployment, even if you have a single machine "cluster". The foundations of Swarm are well-thought and works perfectly on just one machine, if you're not using it because you don't have a "big enough" cluster, you're shooting yourself in the foot.

Kubernetes Basics

How to build up-to-date (container) applications

A system such as Kubernetes can be viewed from different angles. Some think of it in terms of infrastructure, as the successor to OpenStack, although the infrastructure is cloud-agnostic. For others, it is a platform which makes it easier to orchestrate microservice architectures — or cloud-native architectures, as they are called nowadays — to deploy applications more easily, plus making them more resilient and scalable.

By Timo Derstappen

For some people, it is a replacement for automation and configuration management tools – leaving complex imperative deployment tools behind and moving on to declarative deployments, which simplify things but grant full flexibility to developers nonetheless.

Kubernetes not only represents a large projection area. It is currently one of the most active open source projects and many large and small companies are working on it. Under the cover of the Cloud Native Computing Foundation, which belongs to the Linux Foundation, a large community is organizing itself. Of course, the focus is on Kubernetes itself, but other projects such as Prometheus, OpenTracing, CoreDNS and Fluentd are also part of the CNCF by now. Essentially, the Kubernetes project is organized through Special Interest Groups (SIGs). The SIGs communicate via Slack, GitHub and weekly meetings, for everyone to attend.

In this article, the focus is less on the operation and internals of Kubernetes than on the user interface. We explain the building blocks of Kubernetes to set up our own application or build pipelines on a Kubernetes cluster.

Orchestration

The resource distribution on a computer is largely reserved for the operating system. Kubernetes performs a similar role in a Kubernetes cluster. It manages resources such as memory, CPU and storage, and distributes applications and services to containers on cluster nodes. Containers themselves have greatly simplified the workflow of developers and helped them to become more productive. Now Kubernetes takes the containers into production. This global resource management has several advantages, such as the more efficient utilization of resources, the seamless scaling of applications and services, and more importantly a high availability and lower operational costs. For orchestration, Kubernetes carries its own API, which is usually addressed via the CLI **kubectl**.

The most important functions of Kubernetes are:

- Containers are launched in so-called pods.
- The Kubernetes Scheduler assures that all resource requirements on the cluster are met at all times.
- Containers can be found via services. Service Discovery allows cluster distributed containers to be addressed by name.
- Liveness and readiness probes continuously monitor the state of applications on the cluster.
- The Horizontal Pod Scaler can automatically adjust the number of replicas based on different metrics (e. g. CPU).
- New versions can be rolled out via rolling updates.

Basic concepts

The rather rudimentary described concepts below are typically needed to start a simple application on Kubernetes.

- **Namespace**: Namespaces can be used to divide a cluster into several logical units. By default, namespaces are not really isolated from each other. However, there are certain ways to restrict users and applications to certain namespaces.
- **Pod**: Pods represent the basic concept for managing containers. They can consist of several containers, which are subsequently launched together in a common context on a node. These containers always run together. If you scale a pod, the same containers are started together again. A pod is practical in that the user can run processes together; processes which originate from different container images, that is. An example would be a separate process which sends a services logs to a central logging service.In the common context of a pod, container memory can share network and storage. This allows porting applications to Kubernetes which had previously run together in a machine or VM. The advantage is that you can keep the release and development cycles of the individual containers separate. However, developers should not make the mistake of pushing all processes of a machine into a pod at once. As a result, it would lose the flexibility of distributing resources in the cluster evenly and scale them separately.
- **Label**: One or more key/value pairs can be assigned to each resource in Kubernetes. Using a selector, corresponding resources can be identified from these pairs. This means that resources can be grouped by labels. Some concepts such as services and ReplicaSets use labels to find pods.
- **Service**: Cubernetes services are based on a virtual construct – an abstraction, or rather a grouping of existing pods, which are matched using labels. With the help of a service, these pods can then, in turn, be found by other pods. Since pods themselves are very volatile and their addresses within a cluster can change at any time, services are assigned specific virtual IP addresses. These IP address can also be resolved via DNS. Traffic sent to these addresses is passed on to the matching pods.
- **ReplicaSet**: A ReplicaSet is also a grouping, but instead of making pods locatable, it's to make sure that a certain number of pods run in the cluster altogether. A ReplicaSet notifies the scheduler on how many instances of a pod are to run in the cluster. If there are too many, some will be terminated until the designated number is reached. If too few are running, new pods will be launched.
- **Deployment**: Deployments are based on ReplicaSets. More specifically: Deployments are used to manage ReplicaSets. They take care of starting, updating, and deleting ReplicaSets. During an update, deployments create a new ReplicaSet and scale the pods upwards. Once the new pods run, the old ReplicaSet is scaled down and ultimately deleted. A Deployment can also be paused or rolled back.
- **Ingress**: Pods and services can only be accessed within a cluster, so if you want to make a service accessible for external access, you have to use another concept. Inbound objects define which ports and services can be reached externally, but unfortunately: Kubernetes in itself does not have a controller which uses these objects. However, there are some implementations within the community, the so-called ingress controllers. A quite typical Ingress controller is the nginx Ingress Controller.
- **Config Maps and Secrets**: Furthermore, there are two concepts for configuring applications in Kubernetes. Both concepts are quite similar, and typically the

DevOpsCon

Also visit this Session:

Running Kubernetes in Production: A Million Ways to Crash Your Cluster

Henning Jacobs (Zalando SE)

Bootstrapping a Kubernetes cluster is easy, rolling it out to nearly 200 engineering teams and operating it at scale is a challenge. In this talk, we are presenting our approach to Kubernetes provisioning on AWS, operations and developer experience for our growing Zalando developer base. We will walk you through our horror stories of operating 80+ clusters and share the insights we gained from incidents, failures, user reports and general observations. Most of our learnings apply to other Kubernetes infrastructures (EKS, GKE, ..) as well. This talk strives to reduce the audience's unknown unknowns about running Kubernetes in production.

configurations are passed to the pod using either the file system or environment variables. As the name suggests, sensitive data is stored in secrets.

An exemplary application

For deploying a simple application to a Kubernetes cluster, a deployment, a service, and an ingress object is required. In this example, we issue a simple web server which responds with a Hello World website. The deployment defines two replicas of a pod with respectively one container of **giantswarm/helloworld**. Both the deployment and the pods are labeled **helloworld**, while the deployment is located in a default namespace (Listing 1).

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: helloworld
  labels:
    app: helloworld
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: helloworld
  template:
    metadata:
      labels:
        app: helloworld
    spec:
      containers:
      - name: helloworld
        image: giantswarm/helloworld:latest
        ports:
- containerPort: 8080
```

To make the pods accessible in the cluster, an appropriate service needs to be specified (Listing 2). This service is assigned to the default namespace as well and has a selector on the label **helloworld**.

```
apiVersion: v1
kind: Service
metadata:
  name: helloworld
  labels:
    app: helloworld
  namespace: default
spec:
  selector:
    app: helloworld
```

All that is missing now is that the service should be accessible externally. Therefore, the service receives an external DNS entry, whereby the clusters Ingress controller then forwards the traffic, which carries this DNS entry in its host header, to the **helloworld** pods (Listing 3).

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  labels:
    app: helloworld
  name: helloworld
  namespace: default
spec:
  rules:
  - host: helloworld.clusterid.gigantic.io
    http:
      paths:
      - path: /
        backend:
          serviceName: helloworld
          servicePort: 8080
```

Note: Kubernetes itself does not carry its own Ingress controller. However, there are some implementations: nginx, HAProxy, Træfik. Professional tip: If there is a load balancer prior to the Kubernetes cluster, it is usually set up so that the traffic is forwarded to the Ingress controller. The Ingress controller service should then be made available on all nodes via NodePorts. Cloud providers typically use the **LoadBalancer** type. This type ensures that the cloud provider extension of Kubernetes automatically generates and configures a new load balancer.

DevOpsCon

Also visit this Session:

Keep Kubernetes save with Real-Time, Run-Time Container Security

Dieter Reuter (NeuVector Inc.)

Using Kubernetes in production brings great benefits with flexible deployments for scaling applications. But DevOps and security teams are facing new challenges to secure clusters, harden container images and protect production deployments against network attacks from the outside and inside. In this talk we'll cover hot topics like how to secure Kubernetes clusters and nodes, image hardening and scanning, and protecting the Kubernetes network against typical attacks. We'll start with on overview of the attack surface for the Kubernetes infrastructure, application containers, and network followed by a live demo of sample exploits and how to detect them. We'll dig into today's security challenges and present solutions to integrate into your CI/CD workflow and even to protect your Kubernetes workload actively with a container firewall.

These YAML definitions can now be stored in individual files or collectively in a file, and loaded onto a cluster with **kubectl**.

```
kubectl create -f helloworld-manifest.yaml
```

The sample code is on GitHub.

Helm

It is possible to file YAML files together in Helm Charts, which helps to avoid a constant struggle with single YAML files. Helm is a tool for the installation and management of complete applications. Furthermore, the YAML files are also incorporated as templates into the Charts, which makes it possible to establish different configurations. This allows developers to run their application on the same chart in a test enviroment, but with a different configuration in the production enviroment. This means that, if the cluster's operating system is Kubernetes, then Helm is the package management. Although, Helm does need a service called Tiller, which can be installed on the cluster via *helm init*. The following commands can be used to install Jenkins on the server:

```
helm repo update
helm install stable/Jenkins
```

The Jenkins chart will then be loaded from GitHub. There are also so-called application registries, which can manage charts, similar to container images (for example quay.io). Developers can now use the installed Jenkins to deploy their own Helm Charts, although this does require the installation of a Kubernetes-CI-Plug-in for Jenkins. This will result in a new Build Step, which can deploy the Helm Charts. The plug-in automatically creates a Cloud configuration in Jenkins and also configures the login details for the Kubernetes API.

More concepts

Distributed Computing software can be challenging. This is the main reason for Kubernetes, to provide even more concepts, as to simplify the construction of such architectures. In most cases, the modules are special variations of above described resources. It is also possible to use them to configure, isolate or extend resources.

- Job: Starts one or more pods and secures their successful delivery
- Cron Job: Starts a Job in a specific or recurring timeframe
- DaemonSet: Sees to it, that Pods are distributed to all (or only a few determined) nodes.
- PersistentVolume,PersistentVolumeClaim: Definition of the storage medium in the cluster and the assignment to Pods.
- StorageClass: Does define the cluster's available saving options
- StatefulSet: Similar to Replica Sets, it does start a specific amount of Pods. These though do have a specified and identifiable ID, which will still be assigned to the Pod even after a restart or a relocation, which is useful for libraries.
- NetworkPolicy: Allows the definition of a set of rules, which does control the networking attempts in a Cluster.
- RBAC: Role-based access control in a Cluster.
- PodSecurityPolicy: Defines the functionality of certain Pods, for example, which a host's resources can be accessed by a container.
- ResourceQuota: Restricts usage of resources inside a Namespace.
- HorizontalPodAutoscaler: Scales Pods, based on the Cluster's metrics.
- CustomResourceDefinition: Extends and adds a custom object to the Kubernetes AI. With CustomController, these objects can then also be managed within the Cluster (see: Operators)

In this context, one should not forget that the community is developing many tools and extensions for Kubernetes. The Kubernetes incubator currently contains 27 additional repositories and many other software projects offer interfaces for the Kubernetes API or are already equipped with Kubernetes manifestos.

Conclusion

Kubernetes is a powerful tool and the sheer depth of every single concept is just impressive. Though it probably will take some time to get a clearer overview of the tool's possible operations. It's still very important to mention, how all of its concepts are build upon each other so that it is possible to form building blocks, which then can be combined into whatever is needed at the time. This is one of the main strong points Kubernetes has, in contrast to regular frameworks, which abstract run times and processes and press applications into a specific form. Kubernetes grants a very flexible design in this regard. It is a well-rounded package of IaaS and Pass, which can draw upon Google's many years of experience in the field of distributed computing. This experience can also be seen in the project's contributors, who were able to apply their knowledge to it, due to learning from mistakes, which were made in previous projects, like the OpenStack, CloudFoundry and Mesos project. And today Kubernetes is widely spread in its use, all kinds of companies are using it, from GitHub and OpenAI to even Disney.

Timo Derstappen is co-founder of Giant Swarm in Cologne. He has many years of experience in building scalable and automated cloud architectures and his interest is mostly generated by lightweight product, process and software development concepts. Free software is a basic principle for him.

Interview with Nicki Watt, CTO at OpenCredo

Taking the pulse of DevOps: "Kubernetes has won the orchestration war"

Should you pay more attention to security when drafting your DevOps approach? Is there a skills shortage in the DevOps space? Will containers-as-a-service become a thing in 2018? We talked with Nicki Watt, CTO at OpenCredo about all this and more.

By Gabriela Motroc

JAXenter: What are your DevOps predictions for 2018? What should we pay attention to?

Nicki Watt: The increasing adoption of complex distributed systems, underpinned by microservices and serverless architectures is resulting in systems with more unpredictable outcomes. I believe the next wave of DevOps practices and tooling will look to address these challenges by focusing on reliability, as well as gaining more intelligent, runtime insight. I see disciplines like Chaos Engineering, and toolchains optimized for runtime Observability becoming more prevalent.

I also believe there is a very real skills shortage in the DevOps space. This will increasingly incentivize organizations to offload their "DevOps" responsibility to commoditized offerings in the cloud. For example, migrating from bespoke, in-house Kubernetes clusters to a PaaS offering from cloud vendors (e.g. EKS, GKE, AKS).

JAXenter: What makes a good DevOps practitioner?

Nicki Watt: Let's be honest, technical competence is a key factor. To be truly effective, however, you need a combination of technical competence and human empathy. Being able to appreciate the fundamental technical and human concerns of your colleagues goes a long way in helping you to become a key part of a team that can drive and deliver change.

JAXenter: Will DevOps stay as it is now or is there a chance that we'll be calling it DevSecOps from now on?

Nicki Watt: I have always seen security as a core component of any DevOps initiative. As security tools and processes become more API driven and automation friendly, we will begin to see more aspects being incor-

Nicki Watt is a techie at heart and CTO at OpenCredo. She has experience working as an engineer, developer, architect and consultant across a broad range of industries including within Cloud and DevOps. Whether programming, architecting or troubleshooting, her personal motto is "Strive for simple when you can, be pragmatic when you can't". Nicki is also co-author of the book Neo4j in Action, and can be seen speaking at various meetups & conferences.

porated into pipelines and processes. Whatever we call it, as long as we build security in from the beginning, that's all that matters!

JAXenter: Do you think more organizations will move their business to the cloud in 2018?

Nicki Watt: Yes, for a few of reasons, but I shall elaborate on just two.

Security concerns have been a significant factor holding organizations back from adopting the cloud, but this is changing. Education, as well as active steps taken by cloud vendors to address security concerns, have allowed previously security wary organizations to be enticed into action. Additionally, I believe hearing cloud success stories from traditional enterprises (at conferences etc.) acts to remove barriers. It emboldens others in similar situations to (re)consider what benefits it may bring them.

The ability to innovate, experiment and scale quickly is something which the cloud excels at. Whilst running production workloads may still be a step too far for some organizations, many are prepared to start using the cloud for experimentation, and dev/test workloads. As more familiarity and experience is gained, production workloads, in time, will also be conquered.

JAXenter: Will containers-as-a-service become a thing in 2018? What platform should we keep an eye on?

Nicki Watt: I believe so. Managing complex distributed systems is hard. The shortage of good skills, and desire to focus available engineering effort on adding genuine business value, makes CaaS a good option for many organizations.

The key differentiator between CaaS platforms is the orchestration layer and herein lays the choice. In my opinion, all other things considered equal, Kubernetes has won the orchestration war. As part of the CNCF — and backed by a myriad of impressive organizations —, the Kubernetes platform provides a consistent, open, vendor-neutral way to manage & run your workloads. It is also available in various CaaS forms from the major cloud vendors now.

JAXenter: Is Java ideal for microservices developments? Should companies continue to invest resources in this direction?

Nicki Watt: Absolutely, no, maybe … it depends. Any technology choice involves tradeoffs and the language you choose to write your microservices in is no different. One of the benefits of microservices is that you should be able to mix and match. Whatever is most appropriate, and I don't see why Java should not be in the mix.

In its favor, Java has a large ecosystem of supporting tools and frameworks out there, including those supporting microservice architectures (SpringBoot, DropWizard etc). Recruitment wise, Java developers are also far easier to get hold of. It is not however without its critics; too verbose, too slow & heavy on resources, especially for short-running processes. In these cases, maybe an alternative would be better.

The question for me is, what are you optimizing for? Are you planning on running 100's of microservices or 10's? Are you latency, memory or process startup sensitive? What does your workforce and current skill base look like? And a crucial one, especially for enterprises, what freedom are you willing, or not, to give development teams? The answer lies in the grey intersection of the response to questions such as these.

JAXenter: Containers (and orchestration tools) are all the rage right now. Will general interest in containers grow this year?

Nicki Watt: Yes I think so. Containers offer a great simplified packaging and deployment strategy and whilst serverless is also on the charge, I see interest in containers continuing. In terms of handling older applications, not everything has to be implemented in containers; this depends on business objectives and requirements. Sometimes a complete rewrite is required but progression along slightly gentler evolutionary tracks is also a good option.

For example: carve monolithic applications up, implementing only the parts in new tech where it makes sense. Alternatively, merely being able to get out of a data center and into the cloud, even on VM's as a first pass, could yield great business returns too.

JAXenter: What challenges should Kubernetes address in 2018?

Nicki Watt: As Kubernetes-based CaaS offers increase, it would be nice to see the community concen-

DevOpsCon

Also visit this Session:

From Legacy To Cloud

Roland Huß (Red Hat)

Everybody is enthusiastic about the "cloud", but how do I can transfer my old rusty application to this shiny new world? This presentation describes the migration process of a ten-year-old web application to a Kubernetes based platform. The app is written in Java with Wicket 1.4 as the Web framework of choice, running on a plain Jetty 6 servlet engine with Mysql 5.0 as backend. Step by step we will migrate this application to Docker containers, eventually running on a self-provisioned Kubernetes cluster in the cloud. We will hit some stumbling blocks, but still, we will see how this migration could be performed relatively effortless. In the end, we have learned, that "containerisation" does not only make sense for green-field projects but also for older applications to pimp up their legacy.

trating on how the security of the cloud providers is better integrated and offered through the Kubernetes platform.

JAXenter: How will serverless change in 2018? Will it have an impact on DevOps?

Nicki Watt: Adoption-wise serverless is still pretty new, so it's early days to make strong predictions. One obvious way I see it evolving is by supporting broader language and option support. e.g. as already seen by AWS Lambda support for Golang.

I still observe that people have a hope that serverless will usher in a "NoOps" era — i.e. one where they don't have to worry about operations at all — it will magically happen! The reality is that people land up acquiring an "AlternativeOps" model. Serverless can magnify many distributed system challenges; for example, there tend to be more processes than say, compared to a microservices architecture. They also often have a temporal (limited time to run) angle to them. Whilst there may be less low-level config going on, there will be more at the API, inter-process and runtime inspection level (logging, tracing and debugging). I believe more DevOps processes and tooling will need to focus on providing cohesive intelligence and insight into the runtime aspects of such systems.

JAXenter: Will serverless be seen as a competitor to container-based cloud infrastructure or will they somehow go hand in hand?

Nicki Watt: I see them more as options in your architectural toolbox. Each offers a very different architectural approach and style, and have different trade-offs. Sometimes all you will need is a hammer. Other times, a quick-fire nail gun, other times a bit of both.

Context is always key and your resulting architecture should evolve based on questions like Do you need long-running processes? Are you latency and/or cost sensitive? Is this an event-driven system? etc.

Architectures also change and evolve. The only approach I would definitely not recommend is one where a decision to go in some direction is made up front, at a high level, without considering context.

JAXenter: Could you offer us some tips & tricks that you discovered this year and decided to stick to?

Nicki Watt: More a principle than tip or trick per se but one I feel more strongly about as time goes on: "Invest your engineering effort in what matters most and adds value, offload the rest".

Choose to concentrate your engineering resources on work which actually adds business value. Where someone else (cloud provider or SaaS) have competently demonstrated the ability to manage and run complex supporting infrastructure type of resources, and it fits (or you can adjust to make it fit) your requirements, let them do it.

A specific simple example, in this case, is using something like AWS RDS instead of running your own HA RDBMS setup on VMs, but there are many more (K8S clusters, observability platforms etc.). In my opinion, this approach saves time and effort and gives you (and your investors) more bang for your buck than trying to do it yourself.

Thank you very much!

DevOpsCon

Also visit this Workshop:

Kubernetes Workshop

Erkan Yanar (linsenraum.de)

This workshop will be held in german. As a participant of this workshop you only need an SSH client. A prepared server accesses the Kubernetes cluster in which its own namespace is provided. We will deal with the most important Kubernetes objects in order to roll out our own application in the Kubernetes cluster. We will get to know the following objects:

pods
deployments
services
Secrets/Configmaps
PVC/PV

The missing objects are presented. The participants will learn how to roll out their own application in a Kubernetes Cluster. You will also discover why developers do not need more than an access to a Kubernetes Cluster to monitor the application (log/metric/availability monitoring with Prometheus and Elastic)